AIORI HACKATHON 3 · A3-PS001-TC197

Catching ransomware-like behaviour early

Android Ransomware-like Activity Early Warning in a Controlled Lab

Dr. Manonmani S
Anil A
Avinash R

github.com/ANIL6190/A3-PS001-TC197 · October 2026

Abstract

Ransomware locks people out of their own files, and it works fast. Any early warning we can give is worth having.

The problem

Ransomware-like apps write to, rename and replace many files in a short time. Signature checks can miss new variants.

The catch

Heavy file activity is not always an attack. Backups and big edits can look similar, so false alarms matter.

Our idea

Learn what a burst of suspicious file activity looks like from simple counts, then warn and say why.

This is a proof of concept in a controlled lab. No real ransomware, and no encryption of real files.

Background

MITRE ATT&CK lists data encryption for impact as a known Android technique [1]. Android also keeps each app's files in a private sandbox [2], so our test app only ever touches its own dummy files.

Signal	What we count	Why it might help
file_creates	New dummy files	Floods of new files are unusual
file_writes	Write operations	Bursts of writes can come early
file_renames	Renames	Mass renames are a red flag
unique_files	Distinct files touched	Wide spread vs. one-file edit
write_rate	Writes per second	How fast it all happens

These are features we chose. No single one proves ransomware on its own.

Literature review

Source	What it tells us	How we use it
[1] MITRE ATT&CK T1471	Describes how Android ransomware encrypts data to deny access	Grounds the behaviour we imitate; not used as a dataset
[2] Android security docs	Explains app sandboxing and process isolation	Keeps our test app inside its own private files
[3] Sharma et al., 2021	Surveys how Android ransomware is analysed and detected	Why we look at behaviour, not only signatures
[4] ML pipelines survey, 2022	Covers evaluation practice and problems like model drift	Why we keep a test split and write down limits

Takeaway: earlier work backs looking at behaviour. Results on synthetic data still cannot tell us how this performs on real traces.

Proposed solution

1 **Make the data**

Labelled normal and suspicious-like records, with some borderline cases on purpose.

2 **Pick the features**

Per time window: writes, renames, new files, distinct files and writes per second.

3 **Train**

A scikit-learn Random Forest with a fixed seed, so anyone can rerun it.

4 **Explain**

Show the label, a model score (not a guaranteed probability) and the signals behind it.

5 **Check it**

Hold out a test split; report confusion matrix, precision, recall, F1 and false positives.

Built with Python, pandas, scikit-learn and Streamlit. FastAPI and an Android test app are optional extras.

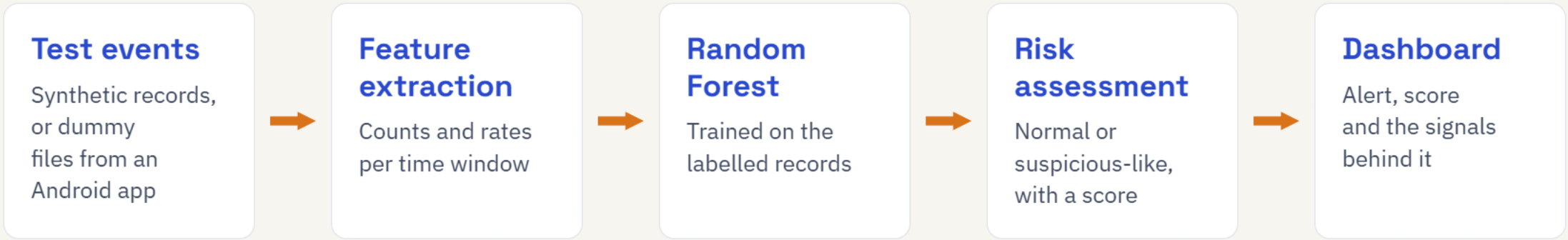
Optimization

Aspect	Before	After	How we check
Detection signal	One threshold on write count	Writes, renames, files and rate used together	Precision and recall on held-out data
Alert usefulness	Only a "suspicious" label	Label, observed signals and limits	Each scenario explains its signal
Evaluation	Training score only	Separate test split and confusion matrix	Report false positives and negatives
Safety	Broad file access	Dummy files in the app sandbox only	Review permissions and file paths

Held-out synthetic split (500 records): this measures our simulation, not real ransomware.

Precision 0.96 · Recall 0.95 · F1 0.96 · False positives 9 · False negatives 13

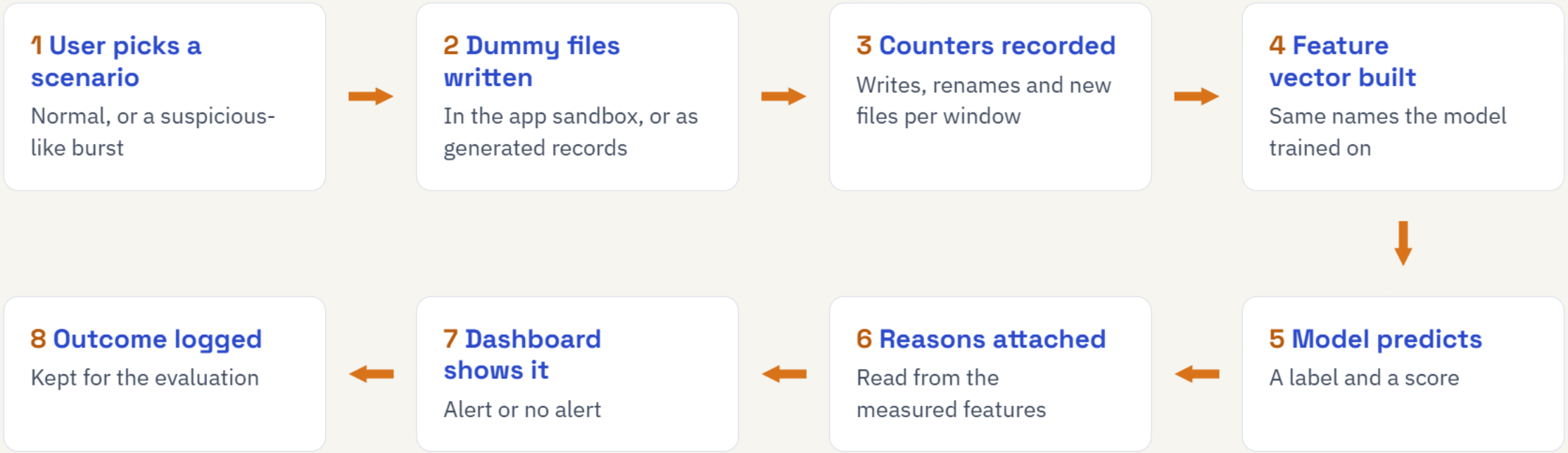
Solution architecture



Only synthetic records and our own app's dummy files go in. Nothing here scans or touches other apps.

The Android app is an extension. The Python pipeline works without it.

Data flow



Only generated counters and dummy-file details. No personal files, no other apps.

Deployment timeline

Phase	What we do	Done when
1. Scope and setup	Agree the safe scope, install tools, generate the synthetic CSV	The data file exists and setup is written down
2. Model	Train the Random Forest and read the confusion matrix	Metrics rerun on the held-out split
3. Demo	Streamlit page with normal and suspicious-like scenarios	Both scenarios run without errors
4. Android extension	Kotlin emulator app writes dummy files and counts actions	Only if the Python demo is stable
5. Package	README, screenshots, limits, references and PDF on GitHub	A fresh clone runs from the README

Python first. Android is a bonus, not a promise.

Bibliography

[1] MITRE ATT&CK. "Data Encrypted for Impact, Technique T1471 – Mobile." attack.mitre.org/techniques/T1471/

[2] Android Open Source Project. "App security best practices." source.android.com/docs/security/best-practices/app

[3] S. Sharma, R. Kumar and C. Rama Krishna. "A survey on analysis and detection of Android ransomware." Concurrency and Computation: Practice and Experience, 33, e6272 (2021). doi.org/10.1002/cpe.6272

[4] "On building machine learning pipelines for Android malware detection: a procedural survey of practices, challenges and opportunities." Cybersecurity (2022). doi.org/10.1186/s42400-022-00119-8

A3-PS001-TC197

Thank you

Questions and discussion

Dr. Manonmani S · Anil A · Avinash R

github.com/ANIL6190/A3-PS001-TC197

A safe prototype for studying early-warning signals. It is not a claim of production-ready ransomware protection.